

CityOS Agentic Stack

Project Overview and Architecture Reference

Winlab Summer 2026

CS3, Center for Smart Streetscapes

June 2026

What We Are Building

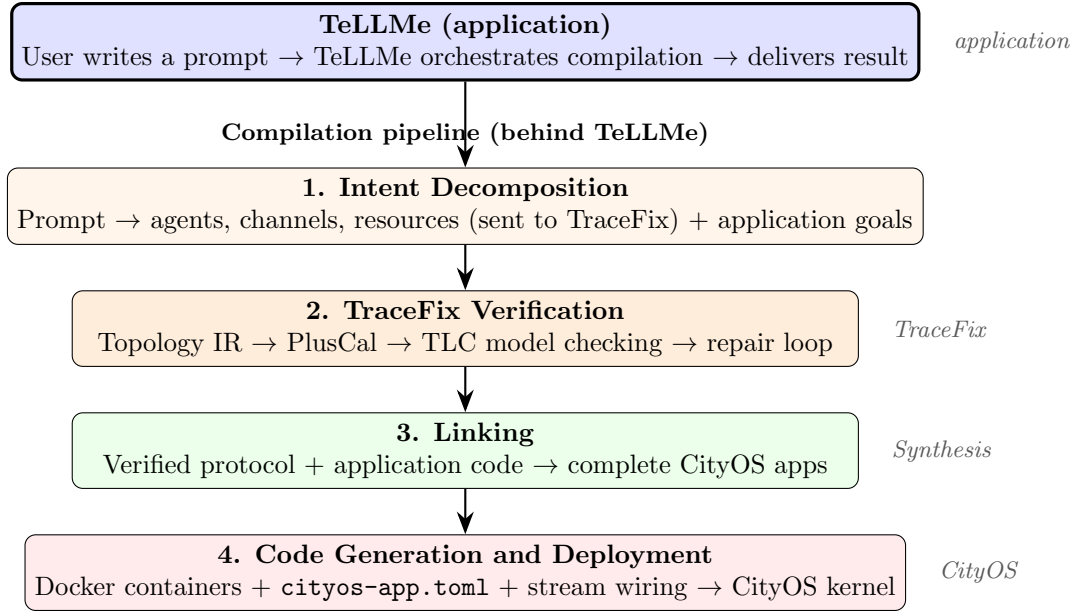
In this system, **applications are prompts**. The user does not write source code. The user writes a natural-language prompt describing what they want (“alert me when a bike and pedestrian are in the same lane”), and the system compiles that prompt into deployed, privacy-preserving CityOS containers with formally verified coordination guarantees.

TeLLMe is the application. It is what users interact with. Behind TeLLMe, a compilation pipeline turns the user’s prompt into running CityOS code. The prompt is the new C. The final output is CityOS application code, Docker containers running on edge nodes, with three properties *guaranteed by the compilation process*:

1. **Privacy**, enforced by CityOS (same-location policy, ephemeral containers, differential privacy).
2. **Distributed safety**, enforced by TraceFix (TLA+ model checking, verified coordination protocols, topology monitoring).
3. **Robustness to failure**, enforced by ConcordFS (durable FIFO channels, checkpointing, crash recovery).

The Compiler Analogy

Just as a C compiler goes through preprocessing, assembly translation, linking, and executable generation, the compilation pipeline behind TeLLMe goes through analogous stages. TeLLMe itself is not one of these stages. TeLLMe is the application that receives the prompt, orchestrates the compilation, and delivers the result.



Core Design Principle

Deterministic checkers verify the output of non-deterministic generators.

The LLM is the non-deterministic producer. Every verifier in the stack (TLC model checker, topology monitor, ConcordFS validators) is deterministic. Both compile-time verification through TLC and runtime verification through the topology monitor are deterministic. There is *no LLM-as-judge* anywhere in the verification path. The checkers are inspectable if-else logic.

Adding agents mid-execution is dangerous because you can no longer guarantee that the communication pattern is verified. All coordination planning happens at compile time via TraceFix.

LLM Models in the Stack

The system uses LLMs at specific stages in the pipeline. Every other component is deterministic. The table below shows exactly where LLMs are used, which model to use, and who is responsible.

Pipeline Stage	What the LLM Model Does	Team
1. Intent Decomposition	Parses user prompt into systems synthesis (agents, channels, resources) and application goals	GPT-4o or Claude 4 Sonnet TeLLMe (Team 4)
2a. IR Synthesis	Generates topology IR JSON from task description	GPT-4o or Claude 4 Sonnet TraceFix (Team 1B)
2b. PlusCal Generation	Generates PlusCal coordination code per agent	GPT-4o or Claude 4 Sonnet TraceFix (Team 1B)
2c. Repair Agent	Revises PlusCal when TLC finds counterexamples	GPT-4o or Claude 4 Sonnet TraceFix (Team 1B)
3. Code Synthesis	Generates CityOS app Python code from verified process bodies	GPT-4o or Claude 4 Sonnet TeLLMe (Team 4)
Runtime (optional)	LLM-powered agents for summarization, classification, Q&A	GPT-4o-mini or Claude Haiku TeLLMe (Team 4)
Simulated User	Generates test prompts for TeLLMe from CARLA scenario state	GPT-4o-mini CARLA (Team 5)

What does NOT use an LLM. ConcordFS (Team 1A), the topology monitor (Team 1C), the TLC model checker, Smart Room drivers and apps (Team 2), and GroundingEval (Team 3) are all deterministic. They never call an LLM. This is by design. The core principle is that deterministic checkers verify the output of non-deterministic generators.

API keys. Teams 1B+1C, 4, and 5 need OpenAI and Anthropic API keys. Create a `.env` file in your repo root.

```
# .env (do NOT commit this file to git)
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...
```

Ask Shuren or Prof. Ortiz for key access. The TraceFix repo already expects this `.env` format. TeLLMe and CARLA should follow the same convention.

Local models. TraceFix is exploring running on Llama 3 locally via Ollama, which would remove the need for API keys during development. If Llama support is available, Shuren will provide setup instructions. The pipeline is designed so that you can swap models by changing a config parameter without modifying pipeline code.

How to call the LLM in your code. Use the `openai` Python library for GPT-4o and the

`anthropic` library for Claude. Both are installed in the TraceFix environment. Here is the minimal pattern.

```
# GPT-4o via OpenAI
from openai import OpenAI
import os
from dotenv import load_dotenv

load_dotenv() # reads .env
client = OpenAI()

response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": "You are a CityOS app generator."},
        {"role": "user", "content": task_description}
    ]
)
result = response.choices[0].message.content

# Claude via Anthropic
from anthropic import Anthropic

client = Anthropic()

response = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=4096,
    messages=[
        {"role": "user", "content": task_description}
    ]
)
result = response.content[0].text
```

TraceFix already wraps these calls in its pipeline code. Study `tracefix/pipeline/` to see how it structures prompts, parses outputs, and handles retries.

Scope for This Summer

Veritas (the auditing and claim-verification layer) is not being handed off to students because the interface is still being iterated on. The “autoencoder” idea for detecting semantic drift between prompt and generated code is acknowledged as open research.

Getting Started. Repositories and Setup

Every team needs these three repositories. Clone all three before starting your team-specific tasks.

Repository 1. CityOS

CityOS is the runtime platform. Every app you build this summer runs on it.

```
git clone https://github.com/Center-for-Smart-Streetscapes-CS3/cityos.git
cd cityos
```

Prerequisites. Go, Python 3, Docker, `protoc`, `just` (command runner). On Mac, use the Dev Container (`.devcontainer/`).

Build and run the echo app (the simplest CityOS app, your “hello world”).

```
just build echo
just run echo
```

The `just build` command generates gRPC code from `apps/sdk/cityos.proto`, builds a Docker image, and saves it to `apps/build/cityos-echo.tar`. The `just run` command starts the Go kernel, which loads the Docker image, spawns the container, and connects via gRPC over a Unix socket. You should see frame output in the terminal.

Repository 2. TraceFix

TraceFix is the formal verification engine. Teams 1B+1C work on it directly, but all teams should understand what it produces.

```
git clone https://github.com/Sensing-And-Reasoning/TraceFix.git
cd TraceFix

python3 -m venv .venv && source .venv/bin/activate
pip install -e ".[test]"
pip install openai anthropic python-dotenv

# Download TLC (the TLA+ model checker, not included in git)
mkdir -p lib
curl -L -o lib/tla2tools.jar \
  https://github.com/tlaplus/tlaplus/releases/download/v1.8.0/tla2tools.jar

# Verify Java 17 is available
java -version # must show 17+

# Create .env with API keys
echo "OPENAI_API_KEY=sk-..." > .env
echo "ANTHROPIC_API_KEY=sk-ant-..." >> .env
```

Run the verification pipeline on a benchmark task.

```
python -m tracefix.pipeline --benchmark 3E --verbose
```

This runs the full pipeline (IR synthesis → PlusCal → TLC model checking → repair → state extraction → prompt generation) on the “Two-Author Research Report” task. The output workspace contains `ir.json`, `Protocol.tla`, `states.json`, and per-agent prompts.

Repository 3. ConcordFS

ConcordFS is the inter-agent communication bus. Team 1A builds it as a CityOS app.

```
git clone https://github.com/jortizcs/concord.git
cd concord
```

This repo may not yet be public at the time you read this. If it is not available, use the TraceFix coordination code (`tracefix/runtime/monitoring/coord.py` and `tracefix/runtime/enforcement/store.py`) as the semantic reference for channel, lock, and counter behavior. ConcordFS implements these same semantics as a persistent CityOS app.

Your First CityOS App. The Echo Example

Every CityOS app follows the same pattern. Study this example line by line before writing your own.

The Python code (apps/echo/echo.py)

```
import asyncio
import logging
from datetime import datetime
from pathlib import Path
from sdk.python.cityos import CityosServicer

class EchoApp(CityosServicer):
    def __init__(self):
        super().__init__()

    async def receive_frame(
        self, stream_name: str, input_path: Path, timestamp: datetime
    ) -> None:
        # Echo the input frame back to the stdout driver
        await self.send_frame(input_path, ["driver:stdout"])

if __name__ == "__main__":
    logging.basicConfig(level=logging.INFO,
        format="%(%asctime)s [%(levelname)s] %(message)s")
    app = EchoApp()
    asyncio.run(app.start())
```

Every app subclasses `CityosServicer` and overrides `receive_frame()`. The kernel calls `receive_frame()` whenever a new frame arrives on one of the app's input streams. The app processes the frame and optionally calls `send_frame()` to emit output. Destinations are strings like `"stream:my-output"` (to another app's input) or `"driver:stdout"` (to a kernel output driver).

The manifest (apps/echo/cityos-app.toml)

```
TarFile = "cityos-echo.tar"
ImageName = "cityos-echo"
ApiLevel = 1
Trusted = true
MinContextSeconds = 2
MaxContextSeconds = 20
InputStreams = ["parkingcam", "parkingcam_video"]
[OutputStreams.stdout]
AllowedReaders = []
```

Field-by-field explanation.

- `TarFile` and `ImageName` identify the Docker image that `just build` produces.
- `ApiLevel = 1` means this is an on-scene, real-time app with ephemeral containers.
- `Trusted = true` gives the app broader stream and driver access permissions.

- `MinContextSeconds` and `MaxContextSeconds` control container rotation. The kernel spawns a fresh container every `MaxContextSeconds` and overlaps it with the old one for at least `MinContextSeconds` so no frames are dropped during the handoff.
- `InputStreams` lists the streams this app subscribes to. The kernel delivers frames from these streams to `receive_frame()`.
- `[OutputStreams.stdout]` declares an output stream named `stdout`. `AllowedReaders = []` means any app can read it (empty ACL = open access).

For persistent apps (ConcordFS, topology monitor, TraceFix planner), add `Persistent = true`. Persistent apps never rotate. They run for the lifetime of the kernel. See `apps/api2-service/cityos-app.toml` as a model.

The kernel config (config/cityos-kernel.toml)

Apps are registered in the kernel config file. Each entry declares an app type and instance name with optional parameter overrides.

```
[apps.echo.default]
# No overrides needed; uses defaults from cityos-app.toml

[apps.eye-in-the-sky.default]
max_context_seconds = "600"
min_context_seconds = "60"
```

Parameters set here become `PARAM_*` environment variables inside the container. The `just run echo` command overrides the entire app map to run only echo.

Creating a new app

There is no scaffold generator. Copy the echo app.

```
cp -r apps/echo apps/myapp
# Edit apps/myapp/myapp.py (rename class, implement receive_frame)
# Edit apps/myapp/cityos-app.toml (change TarFile, ImageName, streams)
# Edit apps/myapp/Dockerfile (if needed)
just build myapp
just run myapp
```

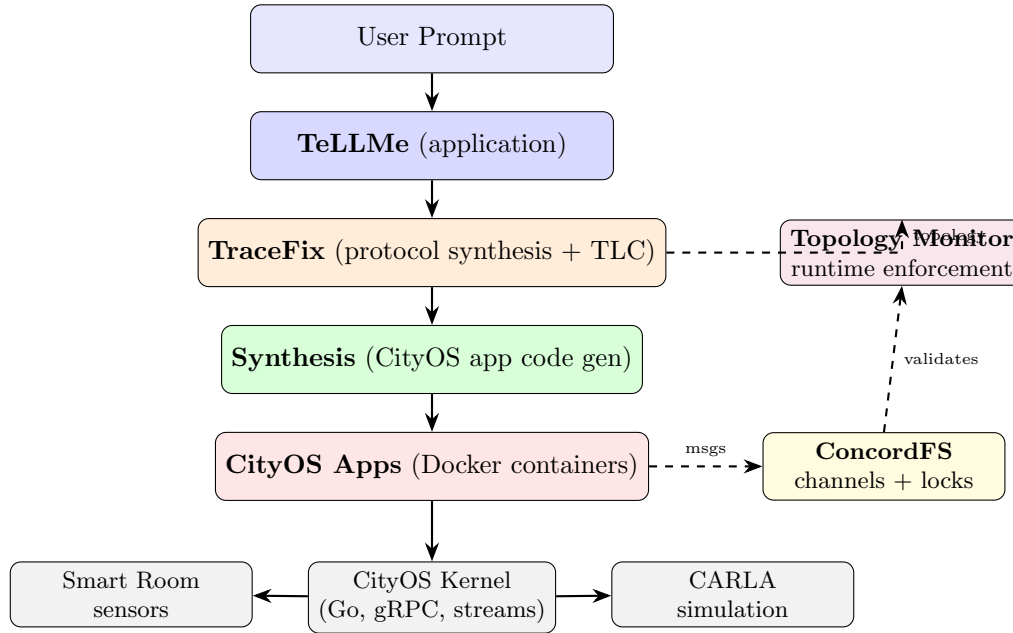

Eye-in-the-Sky. Understanding Detection Output

The **eye-in-the-sky** app runs YOLO object detection + BoT-SORT tracking on camera frames. Its output JSON is the data format that most apps in the stack consume.

```
{
  "input_timestamp_ms": 1718000000123,
  "timestamp_ms": 1718000000456,
  "detected_objects": [
    {
      "box": [120.5, 200.3, 250.1, 400.8],
      "class_id": 0,
      "class_name": "pedestrian",
      "confidence": 0.92,
      "track_id": 7
    },
    {
      "box": [500.0, 180.0, 700.0, 350.0],
      "class_id": 2,
      "class_name": "vehicle",
      "confidence": 0.87,
      "track_id": 12
    }
  ],
  "metrics": {
    "backend": "tensorrt",
    "inference_ms": 8.3,
    "cuda_memory_allocated_mb": 412
  }
}
```

The **box** field contains [x1, y1, x2, y2] pixel coordinates. The **track_id** persists across frames for the same physical object (assigned by BoT-SORT). Smart Room, CARLA, and GroundingEval all work with this format.

Architecture



The CityOS Runtime Constraint

The CityOS kernel is a Go binary with exactly three kinds of entities. **Input drivers** (Go, compiled into the kernel) produce frames from hardware such as RTSP cameras and video files. **Output drivers** (Go, compiled into the kernel) consume frames for display, file output, or streaming. **Apps** (Docker containers) are everything else. There is no plugin system, no middleware hook, and no kernel extension point for custom logic.

This means that ConcordFS, the TraceFix planner, and the topology monitor must all be implemented as **persistent CityOS apps** (`Persistent = true` in their `cityos-app.toml`). Persistent apps run for the lifetime of the kernel and do not rotate through ephemeral containers. The existing API 2 infrastructure follows this same pattern. `api2-service`, `api2-tracking`, and `api3-coordinator` are all persistent CityOS apps, and they are the best reference implementations for how ConcordFS and the topology monitor should be built.

TraceFix topology agents become CityOS apps (Docker containers using `CityosServicer`). TraceFix topology channels become ConcordFS streams, which are CityOS named streams with FIFO ordering. Topology resources such as locks and counters become ConcordFS coordination primitives. TraceFix process bodies are compiled into CityOS app code that calls `receive_frame()` and `send_frame()`. TLC verification happens at compile time before any app is deployed, and the topology monitor validates all runtime messages against the verified topology.

CityOS API Tiers

CityOS implements a *same-location policy*, a physical analog of the web's same-origin policy. Data and computation remain local by default, and broader access is mediated by structured APIs whose privacy protections strengthen as scope expands.

Tier	Name	Description
API 1	On-Scene	Real-time, ephemeral access. Apps run in ephemeral containers with bounded temporal context (minEC / maxEC). Output via localized release only.
API 2	Single-Locality	DP longitudinal statistics at a single location via Toeplitz continual release. Trusted mode uses tracker IDs ($\Delta \approx 9$) and untrusted mode provides formal DP at higher noise.
API 3	Cross-Locality	Cross-location analytics via device-mediated aggregation adapted from W3C Attribution. Per-user IDP budgets are enforced on-device and infrastructure never sees individual trajectories.

API 2 Data Flow (Concrete)

The API 2 privacy pipeline is implemented entirely as CityOS apps. Understanding this data flow helps you see how persistent apps coordinate.

```

Camera -> eye-in-the-sky (YOLO+tracking)
  -> stream:eye-in-the-sky (detection JSON)
  -> api2-tracking (sliding-window person tracker)
  -> stream:tracking-data (tracking snapshots)
  -> api2-query (Rust, Toeplitz DP noise)
  -> stream:api2-releases (noisy DP statistics)
  -> api2-service (release clock + phone broadcasts)
  -> stream:gateway-broadcasts (to phones via HTTP)

```

Each of these (**api2-tracking**, **api2-query**, **api2-service**) is a persistent CityOS app with its own `cityos-app.toml`. Study them in the CityOS repo under `apps/`.

How TraceFix Verification Works

TraceFix (ACM CAIS 2026, Best Paper Award) is a verification-first pipeline for LLM multi-agent coordination.

1. An LLM synthesizes a **protocol topology IR** in JSON from a natural-language task description, defining agents, shared resources (locks, counters), and directed labeled FIFO channels.
2. **PlusCal** coordination logic is generated with one **fair process** per agent, using explicit **send**, **receive**, **acquire_lock**, **release_lock**, and **either/or** nondeterministic branches.
3. **TLC** (the TLA+ model checker) exhaustively checks safety properties including deadlock freedom, mutual exclusion, orphan locks, and channel drainage.
4. When TLC finds violations, the **repair agent** revises PlusCal using the counterexample trace.
5. After verification, **verified process bodies are compiled into per-agent prompts**.

6. A **topology monitor** at runtime rejects out-of-topology coordination operations.

All 48 benchmark tasks reach full TLC verification (100%). 62.5% pass on the first attempt. Topology-monitored execution achieves 89.4% average completion and 81.5% full task completion.

What TraceFix Produces (Verified Workspace)

After running the pipeline on a task, the output workspace contains these files.

```
workspace/3E/
  ir.json           # Topology IR (agents, resources, channels)
  Protocol.tla      # PlusCal source with process bodies
  Protocol.cfg      # TLC configuration
  states.json       # Extracted per-agent state machines
  summary.json      # Repair tracking metadata
  prompts/
    runtime_a/      # Per-agent prompts for enforcement
    runtime_b/      # Per-agent prompts for monitoring
```

Example Topology IR (ir.json)

This is from benchmark task 3E (Two-Author Research Report). Three agents share two locks and communicate through four channels.

```
{
  "agents": [
    {"id": "researcherA"},
    {"id": "researcherB"},
    {"id": "editor"}
  ],
  "resources": [
    {"id": "doc_lock", "type": "Lock"},
    {"id": "ref_lock", "type": "Lock"}
  ],
  "channels": [
    {"id": "resA_to_editor", "from": "researcherA",
      "to": "editor", "labels": ["submit"]},
    {"id": "resB_to_editor", "from": "researcherB",
      "to": "editor", "labels": ["submit"]},
    {"id": "editor_to_resA", "from": "editor",
      "to": "researcherA", "labels": ["revise", "accept"]},
    {"id": "editor_to_resB", "from": "editor",
      "to": "researcherB", "labels": ["revise", "accept"]}
  ]
}
```

The IR has three sections. **agents** lists the concurrent processes. **resources** declares shared locks (mutual exclusion) and counters (counting semaphores). **channels** defines directed FIFO queues between agents, each with a **labels** vocabulary restricting which message types can flow.

ConcordFS as the Communication Substrate

ConcordFS is a **persistent CityOS app** that implements TraceFix’s topology channels as durable, ordered CityOS streams. It runs as a Docker container with `Persistent = true`, subscribes to coordination streams, and validates all inter-agent messages against the loaded topology.

FIFO channels match TraceFix semantics, where send always succeeds and receive blocks when empty. Messages on a channel must use labels from the channel’s declared vocabulary. Senders and receivers are validated against the topology’s `from/to` fields. Locks enforce mutual exclusion and counters are implemented as coordination services. Messages are persisted to durable storage so that crash recovery preserves FIFO ordering.

The closest working reference code is in the TraceFix repo at `tracefix/runtime/monitoring/coord.py` (async channel/lock/counter API) and `tracefix/runtime/enforcement/store.py` (in-memory message, lock, and counter stores). ConcordFS ports these semantics onto CityOS as a persistent app.

Team Map

Team	Project	Members
1A	ConcordFS	Sankeerth + Ryan Baik (HS)
1B+1C	TraceFix + Verified Coordination	Jay + Shreyaa + Rakshan (HS) + Heeun (HS)
2	Smart Room	Crystal + Alan + Annika (HS)
3	GroundingEval	Vikhyat + Aryan (HS)
4	TeLLMe	Sravya (TBC) + Abhishek (HS)
5	CARLA Simulation	Michelle + Ryan Purakal + Vihaan (HS)

Cross-Team Integration Milestones

Week 1 and 2

All teams clone repos and run echo app. TraceFix pipeline reproduced on paper benchmarks. ConcordFS relay app operational. Smart Room driver publishes to a CityOS stream. GroundingEval schemas defined.

Week 2 and 3

ConcordFS validates messages against a loaded topology. TraceFix topology IR extended for CityOS. TeLLMe prompt taxonomy defined. First evaluation scenario labeled.

Week 3 and 4

TraceFix planner runs as a CityOS app. TeLLMe accepts prompts and writes intents. GroundingEval detector evaluator runs on first scenario.

Week 4 and 5

Topology monitor validates ConcordFS traffic. Process body compiler generates working CityOS apps. Smart Room and CARLA produce eye-in-the-sky-compatible streams. GroundingEval 6-scenario suite complete.

Week 5 and 6

TeLLMe generates CityOS apps from verified topologies. End-to-end pipeline operational. GroundingEval connected to TeLLMe answer evaluator.

Week 7 and 8

CARLA simulated user stress-tests the pipeline. Counterexample case studies completed. API 2 integration for longitudinal queries.

Week 9+

Evaluation, demo polish, figures, and handoff.

Background Reading

Before starting your team tasks, work through the following.

1. **How the C compiler works.** Preprocessing, assembly translation, linking, executable generation. This is the mental model for the compilation pipeline behind TeLLMe.
2. **Operating system basics.** Kernel code vs. application space, system calls, and how libraries work. This is how CityOS is designed.
3. **The CityOS repository** at <https://github.com/Center-for-Smart-Streetscapes-CS3/cityos>. Clone it, build and run the echo app, then study `apps/echo/echo.py`, `apps/echo/cityos-app.toml`, and `config/cityos-kernel.toml`.
4. **The TraceFix repository** at <https://github.com/Sensing-And-Reasoning/TraceFix>. Clone it, install dependencies, and run `python -m tracefix.pipeline --benchmark 3E --verbose`. Inspect the output workspace.
5. **The TraceFix paper**, “TraceFix: Repairing Agent Coordination Protocols with TLA+ Counterexamples” (ACM CAIS 2026, Best Paper). Focus on Section 2 (Method) and Section 2.9 (runtime enforcement).
6. **The CityOS paper**, “CityOS: Same-Location Privacy Architecture for Urban Sensing.” Focus on Section 3 (Overview) and Section 4 (Design).

Shuren is the primary technical contact for TraceFix and CityOS implementation details. CityOS runs on a Jetson AGX Orin, and students will have access to a shared Jetson for development and testing.